

Sistemas Distribuidos y Paralelos

Ingeniería en Computación



Modelo de programación sobre memoria compartida

Universidad Nacional de La Plata



Facultad de Informática





1

Modelo de memoria compartida

2

Posix Threads

Estructura de programa Pthreads. Definición de hilos. Creación de hilos. Función de comportamiento de hilos. Espera por finalización de hilos. Sincronización: Mutexes, Variables condición, Barreras. Afinidad. Comentarios finales sobre Pthreads.

3

OpenMP

Modelo Fork-Join. Core elements: Control de flujo paralelo, Entorno de datos y manejo de variables, Trabajo compartido y distribución de la carga, Sincronización, Funciones de biblioteca y variables de entorno. Afinidad. Comentarios finales sobre OpenMP.



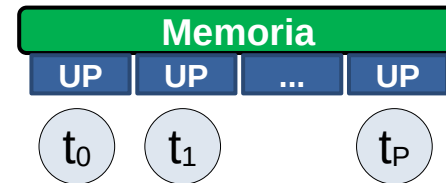
1

Modelo de memoria compartida

Modelo de memoria compartida

4

- Hilos o procesos acceden a un mismo espacio de direcciones y se comunican mediante variables compartidas.
 - **Hilos:** menos costosos para los sistemas operativos.
- Posibles problemas de **condición de carrera**. Solución mediante sincronización:
 - Por Exclusión Mutua
 - Por Condición
- Herramientas para el desarrollo paralelo en este modelo (C/C++ o Fortran):
 - Pthreads
 - OpenMP
 - Otras: Cilk, OpenCL, CUDA, Intel TBB etc.



Modelo de programación sobre memoria compartida

Modelos de programación paralela – Modelos de arquitectura paralela

5

- Puede utilizarse en un **modelo de arquitectura de memoria compartida** con un único espacio de memoria direccionable.
 - En otros modelos de arquitectura, requiere capa de abstracción SSI que genera overhead.

Memoria			
UP	UP	...	UP

Software			
Memoria Compartida (Ej: OpenMP)	Memoria Distribuida (Ej: MPI)	Híbrido (EJ: MPI + OpenMP)	
Trivial (Ej: OpenMP sobre Multicore)	Posible (Ej: MPI sobre multicore)	Posible (extraño) (Ej: MPI + OpenMP sobre multicore)	
(NO ADECUADO) Single System Image(SSl) Overhead	Trivial (Ej: MPI sobre cluster)	Posible (muy extraño) (Ej: MPI + OpenMP sobre cluster monocore)	
(NO ADECUADO) Single System Image(SSl) Overhead	Posible (Ej: MPI sobre cluster multicore)	Trivial (Ej: MPI + OpenMP sobre cluster multicore)	

Hardware	Memoria Compartida (Ej: multicore)
	Memoria Distribuida (Ej: cluster monocore ¹)
	Híbrido (Ej: cluster multicore)

¹Actualmente, los clusters monocore cayeron en desuso pero se mencionan para ejemplificar.

Modelo de programación sobre memoria compartida

6



NO confundir el concepto de **variable global** con el de **variable compartida**

- **Variables globales en lenguajes imperativos: mecanismo de comunicación entre programa principal, módulos/subrutinas (procedures o funciones),** definidas en un área de programa visible por estos.
 - **Desaconsejado** en el desarrollo de **aplicaciones secuenciales** (por legibilidad, portabilidad, modularidad, mantenibilidad, etc.).
- **Variables compartidas: estado compartido** de cómputo concurrente entre hilos.
 - Son parte del modelo
- La confusión en lenguaje C:
 - No existe un área para definir variables compartidas y se definen como globales.
 - Bibliotecas definen hilos como funciones.



Modelo de programación sobre memoria compartida

7



Una variable compartida NO es una variable global
Un hilo NO es una función o un procedure

```
Process P[id:0 to P-1]{  
    ...  
}
```

- Conceptualmente:
 - **Hilos no son funciones (recordar Concurrencia y Paralelismo) y no reciben** variables compartidas por **parámetros** (porque son compartidas!!!).
 - Recordar **ADA: Task** no son funciones ni procedures.
 - El código de los **procesos o hilos no es portable** sin el contexto del proceso que los contiene (modularidad SO), quien es el que provee las variables compartidas.
- Analogía con la Programación Orientada a Objetos:
 - Variables de instancia: variables compartidas por todos los métodos de un objeto.
 - Los métodos no reciben variables de instancia por parámetro (lenguajes POO tampoco lo permiten).
 - El código de los métodos no es portable sin el contexto del objeto que los contiene, quien es el que provee de las variables de instancia.

Modelo de programación sobre memoria compartida

8

Imperativo (secuencial)

```
program Hello;  
  var v: Integer;
```

```
procedure Proc()  
begin  
  v := v + 1;  
end;
```

Pascal

```
function f():integer  
begin  
  f:= v + 1;  
end;  
...
```

```
#include ...  
int v;
```

```
void f(){  
  v++;  
}
```

C

Imperativo
Esto está mal



POO
Esto está bien



POO (secuencial)

```
public class Clase{  
  private int v;  
  
  public void metodo(){  
    v++;  
  }  
  ...  
}
```

JAVA

```
Object subclass: Clase[  
  |v|
```

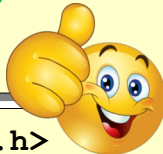
```
Clase » metodo() [  
  v:= v + 1;  
  ]  
  ...  
]
```

Smalltalk



Concurrente

```
int v;  
Process P[id:0 to P-1]{  
  atomic(v++);  
  ...  
}
```



```
#include<pthread.h>  
int v;
```

```
void* f(void* arg){  
  atomic(v++);  
  ...  
}
```

```
main() {  
  ...  
  pthread_create(..., &f, ...);  
  ...  
}
```

Concurrente
Esto está bien

C (Pthreads)

Estructura de programa
variable

subrutina(sin parámetros)
Modifica variable

En todos los casos, son estructuras de programa que definen una variable libre que se modifica por una subrutina directamente, sin parámetros. En imperativo está mal, en POO y Concurrente está bien. La **diferencia** está en el **paradigma** (Imperativo ≠ POO ≠ Concurrente)

Posix Threads

Estructura de programa Pthreads

Definición de hilos

Creación de hilos

Función de comportamiento de hilos

Espera por finalización de hilos

Sincronización:

Mutexes

Variables condición

Barreras

Afinidad

Comentarios finales sobre Pthreads

Pthreads

10

- **Pthread¹ (Posix threads** - estándar Posix 1.0c **1995**) API para programación multihilo sobre memoria compartida multiplataforma
- Objetivo inicial: facilitar programación multihilo (Multitarea: Linux/Windows)
- Conjunto de funciones para:
 - Gestionar hilos (threads)
 - Sincronizar hilos: Mutexes, Variables condición, Barreras
 - Interactuar con la API Semaphore (no es parte del estándar)
- Incluida en el compilador GCC:
 - Cabecera de programa C: `#include<pthread.h>`
 - Compilación: `gcc -pthread fuente.c -o ejecutable`

¹<https://computing.llnl.gov/tutorials/pthreads/>

Posix Threads

Estructura de programa Pthreads

Definición de hilos

Creación de hilos

Función de comportamiento de hilos

Espera por finalización de hilos

Sincronización:

Mutexes

Variables condición

Barreras

Afinidad

Comentarios finales sobre Pthreads

Pthreads

Estructura de programa Pthreads

- Programa **main**: proceso inicial.
 - Gestiona hilos
- Un programa Pthreads 4 pasos:
 - 1) Definir Hilos
 - 2) Crearlos
 - 3) Implementar la función de comportamiento del Hilo
 - 4) El proceso inicial espera que los hilos terminen su ejecución

```
#include<pthread.h>
//Variables compartidas
//Mecanismos de sincronización

//Función de comportamiento del hilo
void* f(void* arg){
...
pthread_exit(NULL);
}

int main(int argc, char*argv[]){
//Definir un hilo
pthread_t miHilo;
...
//Crear el hilo
pthread_create(&miHilo,&attr,&f,&arg);
...
//Esperar que el hilo finalice
pthread_join(&miHilo,NULL);

return 0;
}
```


Posix Threads

Estructura de programa Pthreads

Definición de hilos

Creación de hilos

Función de comportamiento de hilos

Espera por finalización de hilos

Sincronización:

Mutexes

Variables condición

Barreras

Afinidad

Comentarios finales sobre Pthreads

Pthreads

Definición de hilos

14

- Hilos representados mediante el tipo de datos:

```
pthread_t
```

- Pueden definirse individualmente:

```
pthread_t miHilo1;  
pthread_t miHilo2;
```



Concurrencia y paralelismo

```
Process P{  
    ...  
}
```

- O como arreglos de hilos (varios hilos con el mismo comportamiento):

```
pthread_t misHilos[N];
```



Concurrencia y paralelismo

```
Process P[id:0 to P-1]{  
    ...  
}
```

Posix Threads

Estructura de programa Pthreads

Definición de hilos

Creación de hilos

Función de comportamiento de hilos

Espera por finalización de hilos

Sincronización:

Mutexes

Variables condición

Barreras

Afinidad

Comentarios finales sobre Pthreads

Pthreads

Creación de hilos

16

- Los hilos se crean y ponen en ejecución mediante la función:

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine)(void*), void *arg);
```

Parámetros:

- **thread:** puntero al hilo definido con pthread_t.
- **attr:** atributos de configuración del hilo
- **No utilizar, este parámetro. Pasar NULL.** 
- **start_routine:** puntero a la función que implementa el comportamiento del hilo.
- **arg:** argumento pasado como parámetro a la función que implementa el comportamiento del hilo.

Valor de retorno:

- **pthread_create** retorna **cero** si tiene **éxito**, sino retorna un código de error.

Si pthread_create tiene éxito, el hilo se ejecuta inmediatamente, invocando de forma automática a la función que implementa su comportamiento.

Posix Threads

Estructura de programa Pthreads

Definición de hilos

Creación de hilos

Función de comportamiento de hilos

Espera por finalización de hilos

Sincronización:

Mutexes

Variables condición

Barreras

Afinidad

Comentarios finales sobre Pthreads

- Prototipo específico para la función que implementa el comportamiento de hilos:

```
void* funcion(void *arg);
```

Parámetros:

- **arg**: puntero sin tipo (**requiere casting**). Puede apuntar a:
 - 1) Una variable de un tipo simple
 - 2) Una estructura: **NO USAR en el modelo de memoria compartida!!!**



Sin valor de retorno.

- Hilos con el mismo comportamiento reciben en la función pthread_create el mismo puntero a función y cada uno ejecuta su instancia.

```
void* funcionA(void *arg){  
...  
}
```

```
pthread_create(&T1, ..., &funcionA,...);  
pthread_create(&T2, ..., &funcionA,...);  
pthread_create(&T3, ..., &funcionB,...);
```

```
void* funcionB(void *arg){  
...  
}
```



En el **modelo de memoria compartida** con **Pthreads**, asumimos que la función del comportamiento de un hilo **recibe un id**.

NO recibe otros parámetros ni tampoco referencias a variables compartidas.

- Pthreads se utiliza en otros ámbitos diferentes al modelo de programación sobre memoria compartida. Por ejemplo: redes o procesamiento gráfico, donde pueden crearse hilos que no comparten variables.
- En ese contexto, el parámetro de la función de comportamiento del hilo puede referenciar una **variable NO compartida** (por ejemplo: una variable dentro de función).

NO USAR en el modelo de programación sobre memoria compartida



```
<TipoDeRetorno> funcionDeAplicacion(<argumentos>) {  
    Tipo miVariableLocal;  
    ...  
    pthread_create(&miHilo, &attr, &funcion, (void*) &miVariableLocal);  
    ...  
}
```

- Pthreads no tiene una forma de identificar los hilos.
- El programador debe identificarlos unívocamente pasando un identificador como parámetro a la función que implementa el comportamiento del hilo.

```
...
void* funcion(void *arg){
    int id=*(int*)arg;
    ...
}
...
int main(int argc, char* argv[]){
    pthread_t misHilos[2];
    int id1,id2;
    ...
        id1=1;
        pthread_create(&misHilos[id],NULL,&funcion,(void*)&id1);
        id2=2;
        pthread_create(&misHilos[id],NULL,&funcion,(void*)&id2);
    ...
}
```

- **Error típico:**

```
...  
void* funcion(void *arg){  
    int id=*(int*)arg;  
...  
}  
...  
int main(int argc, char* argv[]){  
    pthread_t misHilos[T];  
...  
    for(int id=0;id<T;id++)  
        pthread_create(&misHilos[id],NULL,&funcion,(void*)&id);  
...  
}
```



El argumento id es un puntero al índice del for. En el for podría modificarse el valor de la variable id antes que el hilo creado la lea. Como consecuencia, puede haber más de un hilo con el mismo id.

- Solución 1:

```
void* funcion(void *arg){  
    int id=*(int*)arg;  
    ...  
}
```



```
int main(int argc, char* argv){  
    pthread_t misHilos[T];  
    int threads_ids[T];  
    ...  
    for(int id=0;id<T;id++){  
        threads_ids[id]=id;  
        pthread_create(&misHilos[id],NULL,&funcion,(void*)&threads_ids[id]);  
    }  
    ...  
}
```

- Solución 2:

```
void* funcion(void *arg){  
    int id=(int*)arg;  
    ...  
}
```



```
int main(int argc, char* argv){  
    pthread_t misHilos[T];  
    ...  
    for(int id=0;id<T;id++){  
        pthread_create(&misHilos[id],NULL,&funcion,(void*)id);  
    }  
    ...  
}
```

id es un valor, no un puntero!!!
Pasar un entero y decirle que es una dirección de memoria.
(El compilador gcc retorna 2 warnings que deben ignorarse)

Pthreads

Función de comportamiento

23

- La función de comportamiento de hilos, al finalizar, debe invocar la función:

```
void pthread_exit(void *value_ptr);
```

Parámetros:

- value_ptr**: valor que puede ser **NULL** o un valor que será enviado a cualquier proceso/hilo que espere por su finalización (en concordancia con la función **pthread_join**).

Sin valor de retorno.



```
...  
void* funcion(void *arg) {  
    int ret;  
  
    ...  
    pthread_exit(&ret);  
}  
...
```

Modelo de memoria compartida
No usar value_ptr para retornar resultados!!!

Sólo para control (éxito o no de la función)!!!



```
...  
void* funcion(void *arg) {  
    ...  
    pthread_exit(NULL);  
}  
...
```

Posix Threads

Estructura de programa Pthreads

Definición de hilos

Creación de hilos

Función de comportamiento de hilos

Espera por finalización de hilos

Sincronización:

Mutexes

Variables condición

Barreras

Afinidad

Comentarios finales sobre Pthreads

- El proceso principal (**main**), debe esperar a que los hilos finalicen con la función **pthread_join**:

```
int pthread_join(pthread_t thread, void **value_ptr);
```

Parámetros:

- **thread**: hilo del cual se espera finalización.
- **value_ptr**: valor retornado por **pthread_exit** del hilo anterior. Si no es necesario recibir valor de control es **NULL**.

Valor de retorno:

- Retorna **cero** si tiene **éxito**, sino retorna un código de error.

Pthreads

Espera por finalización

26

Espera sobre hilos individuales

```
void* funcionT1(void *arg){
...
pthread_exit(NULL);
}

void* funcionT2(void *arg){
int success;
...
pthread_exit(&success);
}

int main(int argc, char* argv[]){
pthread_t T1,T2;
int success;
pthread_create(&T1,...,&funcionT1,...);
pthread_create(&T2,...,&funcionT2,...);

pthread_join(T1,NULL);
pthread_join(T2, (void**)&success);
...
}
```

Espera sobre un arreglo de hilos

```
void* funcionT2(void *arg){
int success;
...
pthread_exit(&success);
}

...

int main(int argc, char* argv[]){
pthread_t misThreads[N];
int success;
for(i=0;i<N;i++){
...
pthread_create(&misThreads[i],...,&funcion,...);
}

for(i=0;i<N;i++){
pthread_join(misThreads[i], (void**)&success);
}
...
}
```

Solo para Control
NO RETORNAR RESULTADOS!!!



Posix Threads

Estructura de programa Pthreads

Definición de hilos

Creación de hilos

Función de comportamiento de hilos

Espera por finalización de hilos

Sincronización:

Mutexes

Variables condición

Barreras

Afinidad

Comentarios finales sobre Pthreads

Pthreads

Sincronización

28

- Mecanismos de sincronización en Pthreads:
 - Mutexes
 - Variables condición
 - Barreras
- Es posible implementar semáforos mediante la biblioteca *semaphore* pero no es parte del estándar.

Pthreads

Sincronización

29

- Mecanismos de sincronización en Pthreads:

- Mutexes
- Variables condición
- Barreras

- Estructura general de programa:

- 1) Definirlo (compartido)
- 2) Inicializarlo
- 3) Utilizarlo
- 4) Destruirlo

```
#include<pthread.h>

//Variables compartidas
//Mecanismos de sincronización

void* f(void* arg){
    //Uso de sincronización
    ...
    pthread_exit(NULL);
}

int main(int argc, char*argv[]){
    //Inicialización de mecanismos de sincronización
    ...
    pthread_create(...);
    ...
    pthread_join(...);
    //Destrucción de mecanismos de sincronización

    return 0;
}
```

Posix Threads

Estructura de programa Pthreads

Definición de hilos

Creación de hilos

Función de comportamiento de hilos

Espera por finalización de hilos

Sincronización:

Mutexes

Variables condición

Barreras

Afinidad

Comentarios finales sobre Pthreads

Pthreads

Sincronización - Mutexes

31

- Un **mutex**, abreviación de ***mutual exclusion***, para sincronización por exclusión mutua.
- Dos posibles estados:



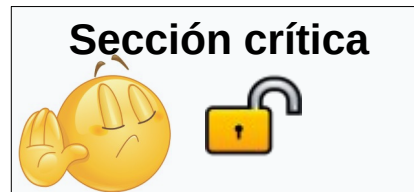
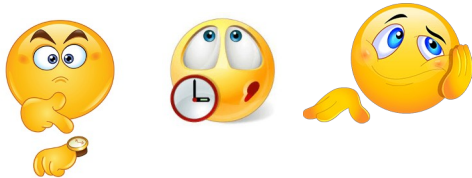
Desbloqueado/Unlocked: Libre.

- Puede ser apropiado por un proceso a la vez.



Bloqueado/Locked: Apropriado por un hilo.

- Si otro hilo intenta apropiarlo quedará en una cola de espera.
- Cuando se libera, se elige un hilo de la cola y se le entrega el mutex.
- Si no hay hilos encolados el mutex cambia a estado desbloqueado.



- Se representan con variables del tipo:

```
pthread_mutex_t
```

- Pueden definirse individualmente:

```
pthread_mutex_t miMutex1;  
pthread_mutex_t miMutex2;
```

- O como arreglos de mutexes:

```
pthread_mutexes_t misMutexes[N];
```


- Inicialización:

- Estática:

```
pthread_mutex_t miMutex = PTHREAD_MUTEX_INITIALIZER;
```

- Dinámica:

```
int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *m_attr);
```

Parámetros:

- **mutex**: puntero al mutex.
- **m_attr**: atributos del mutex. Estructura de tipo **pthread_mutexattr_t** que contiene tres atributos:

Protocol: protocolo para prevención de inversión de prioridades.

Prioceiling: límite de prioridad de un mutex.

Process-shared: especifica el uso compartido de un mutex.

No usar con las aplicaciones paralelas, este parámetro será **NULL**



Valor de retorno:

- **pthread_mutex_init** retorna **cero** si tiene **éxito**, sino retorna un código de error.

- Dos funciones básicas:

- Adquirir un mutex:

```
int pthread_mutex_lock(pthread_mutex_t *mutex);
```

- Si el mutex está desbloqueado, el propietario será el hilo que invoca a la función. Cualquier otro hilo que invoque a la función quedará encolado.

- Liberar un mutex:

```
int pthread_mutex_unlock(pthread_mutex_t *mutex)
```

Parámetros (ambas funciones):

- **mutex:** puntero al mutex.

Valor de retorno (ambas funciones):

- Retornan **cero** si tienen **éxito**, sino retornan un código de error.

- Adicionalmente, pthreads provee la función:

```
int pthread_mutex_trylock(pthread_mutex_t *mutex);
```

Parámetros:

- **mutex**: puntero al mutex.

Valor de retorno:

- Si el hilo puede adquirir el mutex será el propietario y la función retorna **cero**.
- Si el hilo no puede adquirir el mutex no se demora, continúa su ejecución y la función retorna un valor **distinto de cero**.

- Uso habitual:

```
if (pthread_mutex_trylock(&miMutex) == 0) {  
    //Sección crítica  
    pthread_mutex_unlock(&miMutex);  
}
```

- Una vez que un mutex deja de usarse debe ser destruido con la función:



```
int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

Parámetros:

- **mutex**: puntero al mutex.

Valor de retorno:

- pthread_mutex_destroy retorna **cero** si tiene **éxito**, sino retorna un código de error.

Posix Threads

Estructura de programa Pthreads

Definición de hilos

Creación de hilos

Función de comportamiento de hilos

Espera por finalización de hilos

Sincronización:

Mutexes

Variables condición

Barreras

Afinidad

Comentarios finales sobre Pthreads

- **Sincronización por condición:** Demorar la ejecución de un hilo a la espera de la ocurrencia de alguna condición:
 - Que haya datos disponibles para consumir.
 - Que un recurso quede libre.
 - Que cierto estado del programa cambie.
- Otro hilo enviará una señal al hilo demorado cuando la ejecución se cumpla.
- **Variables condición:** permiten demorar y señalar eventos.
- Cada variable condición tiene asociada una cola de espera.
- Cada variable condición debe utilizarse en conjunto con un mutex.



- Se representan con variables del tipo:

```
pthread_cond_t
```

- Pueden definirse individualmente:

```
pthread_cond_t c1;  
pthread_cond_t c2;
```

- O como arreglos de variables condición:

```
pthread_cond_t cs[N];
```

- Inicialización:

- **Estática:** `pthread_cond_t c = PTHREAD_COND_INITIALIZER;`

- **Dinámica:**

```
int pthread_cond_init(pthread_cond_t *restrict cond, const pthread_condattr_t *restrict cond_attr);
```

Parámetros:

- **cond:** puntero a la variable condición.
- **cond_attr:** parámetro para personalizar atributos de la variable condición. Una estructura de tipo **pthread_condattr_t** que permite limitar el alcance de la variable condición y solo puede modificarse mediante funciones:

`pthread_condattr_getpshared`

`pthread_condattr_setpshared`

No usar en las aplicaciones paralelas, este parámetro será **NULL**



Valor de retorno:

- **pthread_cond_init** retorna **cero** si tiene **éxito**, sino retorna un código de error.

Pthreads

Sincronización – Variables condición – Funciones básicas

- Tres funciones básicas:

- Demorar/Dormir un hilo sobre una variable condición:

```
int pthread_cond_wait(pthread_cond_t *restrict cond, pthread_mutex_t * mutex);
```

- Despertar un hilo demorado en una variable condición:

```
int pthread_cond_signal(pthread_cond_t *cond);
```

- Despertar todos los hilos demorados en una variable condición:

```
int pthread_cond_broadcast(pthread_cond_t *cond);
```

Parámetros (de las tres funciones):

- **cond**: puntero a la variable condición.
- **mutex**: (sólo función `pthread_cond_wait`) puntero a un mutex asociado.

Valor de retorno (de las tres funciones):

- Retornan **cero** si tienen **éxito**, sino retornan un código de error.

Mutex?



- `pthread_cond_wait` necesita un **mutex** asociado a la variable condición:

```
pthread_mutex_lock(&mutex);  
    if("NO cumple condición")  
        pthread_cond_wait(&c, &mutex);  
    ...  
pthread_mutex_unlock(&mutex);
```

Monitores



- Si el hilo debe demorarse, se libera el mutex automáticamente.
 - Cuando el hilo despierte, tomará el mutex automáticamente.
- `pthread_cond_signal` y `pthread_cond_broadcast` no tienen asociado un mutex, pero lo habitual es usarlo en consonancia con `pthread_cond_wait`:

```
pthread_mutex_lock(&mutex);  
    ...  
    pthread_cond_signal(&c);  
    ...  
pthread_mutex_unlock(&mutex);
```

```
pthread_mutex_lock(&mutex);  
    ...  
    pthread_cond_broadcast(&c);  
    ...  
pthread_mutex_unlock(&mutex);
```

- Una vez que una variable condición deja de usarse debe ser destruida con la función:



```
int pthread_cond_destroy(pthread_cond_t *cond);
```

Parámetros:

- **cond:** puntero a la variable condición.

Valor de retorno:

- **pthread_cond_destroy** retorna **cero** si tiene **éxito**, sino retorna un código de error.

Posix Threads

Estructura de programa Pthreads

Definición de hilos

Creación de hilos

Función de comportamiento de hilos

Espera por finalización de hilos

Sincronización:

Mutexes

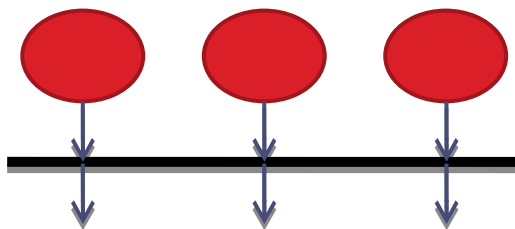
Variables condición

Barreras

Afinidad

Comentarios finales sobre Pthreads

- Barreras: conjunto de hilos/procesos que esperan para poder continuar su ejecución.



- Pthreads provee de una implementación sencilla y eficiente de las barreras, evitando que el programador tenga que implementarlas.

- Se representan con variables del tipo:

```
pthread_barrier_t
```

- Pueden definirse individualmente:

```
pthread_barrier_t barrera1;  
pthread_barrier_t barrera2;
```

- O como arreglos de barreras:

```
pthread_barrier_t barreras[N];
```

- Inicialización mediante la función:

```
int pthread_barrier_init(pthread_barrier_t * barrier, const pthread_barrierattr_t *b_attr, unsigned count);
```

Parámetros:

- **barrier**: puntero a la barrera.
- **b_attr**: parámetro para personalizar los atributos de la barrera. Una estructura de tipo **pthread_barrierattr_t** que permite definir atributos con las características de la barrera y solo puede modificarse mediante funciones:
 pthread_barrierattr_getpshared
 pthread_barrierattr_setpshared
 No usar en las aplicaciones paralelas, este parámetro será **NULL** 
- **count**: número de hilos que deben llegar a la barrera para poder continuar.

Valor de retorno:

- **pthread_barrier_init** retorna **cero** si tiene **éxito**, sino retorna un código de error.

- Un hilo usa la barrera mediante la función:

```
int pthread_barrier_wait(pthread_barrier_t *barrier);
```

Parámetros:

- **barrier**: puntero a la barrera.

Valor de retorno:

- **pthread_barrier_wait** retorna **cero** si tiene **éxito**, sino retorna un código de error.

- Si el número de hilos en la barrera es menor al especificado en el parámetro **count** de la función de inicialización, el hilo se demora.
- En caso contrario, todos los hilos demorados en la barrera continuarán la ejecución.

- Una vez que una barrera deja de usarse debe ser destruida con la función:



```
int pthread_barrier_destroy(pthread_barrier_t *barrier);
```

Parámetros:

- **barrier:** puntero a la barrera.

Valor de retorno:

- pthread_barrier_destroy retorna **cero** si tiene **éxito**, sino retorna un código de error.

- En aplicaciones con varias etapas de ejecución, NO crear Hilos por cada etapa. Crear hilos una única vez y separar cada etapa por una barrera.



```
int main(int argc, char* argv[]){
...
for(int id=0;id<T;id++){
    pthread_create(&h[id],NULL,&etapa1,(void*)id);
}
...
for(int id=0;id<T;id++){
    pthread_create(&h[id],NULL,&etapa2,(void*)id);
}
...
}
```

```
void* etapas(void* arg){
...
    etapa1();
    pthread_barrier_wait(&barrera);
    etapa2();
    pthread_barrier_wait(&barrera);
...
}
```



```
int main(int argc, char* argv[]){
...
for(int id=0;id<T;id++){
    pthread_create(&h[id],NULL,&etapas,(void*)id);
}
...
}
```



Para el sistema operativo el costo de crear hilos una y otra vez es mayor que dormirlos y despertarlos.

Posix Threads

Estructura de programa Pthreads

Definición de hilos

Creación de hilos

Función de comportamiento de hilos

Espera por finalización de hilos

Sincronización:

Mutexes

Variables condición

Barreras

Afinidad

Comentarios finales sobre Pthreads

Pthreads

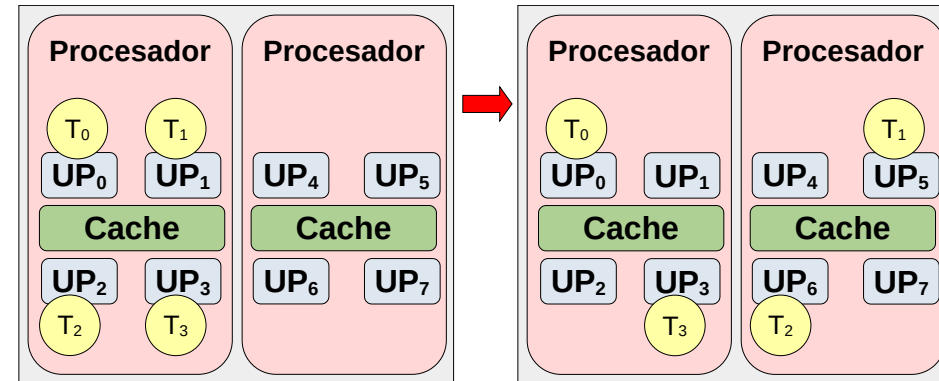
Afinidad

52

Afinidad: Elegir la unidad de procesamiento donde debe ejecutar un hilo.

¿Por qué usar afinidad?

- Aplicaciones intensivas en memoria pueden obtener mejor rendimiento al ejecutar con menos hilos que unidades de procesamiento disponibles.
 - La asignación debe estar balanceada respecto a recursos compartidos.
- El sistema operativo puede mapear de forma desbalanceada.
- El programador puede volver a mapear
 - **Afinidad.**



Asignación de hilos
no balanceada.

Asignación de hilos
balanceada.



Pthreads

Afinidad

53

- Encabezado para utilizar afinidad en pthreads:

```
#define _GNU_SOURCE
#include<sched.h>
```

- Cambiamos la afinidad de un hilo con la función:

```
int pthread_setaffinity_np(pthread_t thread, size_t cpusetsize, const cpu_set_t *cpuset)
```

Parámetros:

- **thread:** hilo al que debe cambiarse la afinidad.
- **cpusetsize:** tamaño en bytes del CPU Set.
- **cpuset:** conjunto de cores de la arquitectura.

Valor de retorno:

pthread_setaffinity_np retorna **cero** si tiene **éxito**, sino retorna un código de error.

Pthreads

Afinidad – CPU set

54

- El parámetro **cpuset** es de tipo **cpu_set_t**, una estructura similar a:

cpuset	0	1	0	1	0	1	0	1
Core	0	1	2	3	4	5	6	7

En este ejemplo, el hilo puede ejecutar en los cores impares

- Las variables de tipo **cpu_set_t** no pueden modificarse directamente, para esto se utiliza la macro:

```
CPU_SET(int cpu, cpu_set_t *set)
```

Pthreads

Afinidad

55

- La función `pthread_setaffinity_np` puede invocarse de dos formas diferentes:

En el proceso que crea el hilo

```
#define _GNU_SOURCE
#include<pthread.h>
#include<sched.h>
...
int main(int argc, char* argv[]){
    pthread_t h; int idH=1;
    cpu_set_t mask;

    pthread_create(&h, NULL, &funcionHilo, (void*)&idH);

    // Pone en cero la mascara
    CPU_ZERO(&mask);
    // Pone en uno el bit del core 3 en la máscara
    CPU_SET(3, &mask);
    // Cambia la afinidad al hilo
    pthread_setaffinity_np(h, sizeof(cpu_set_t), &mask);

    pthread_join(hilo, NULL);
    ...
}
```

En el propio hilo

```
#define _GNU_SOURCE
#include<pthread.h>
#include<sched.h>

void* funcionHilo(void *arg){
    // Obtiene el descriptor del hilo
    pthread_t h=pthread_self();
    // Pone en cero la mascara
    CPU_ZERO(&mask);
    // Pone en uno el bit del core 3 en la máscara
    CPU_SET(3, &mask);
    // El hilo se cambia su afinidad
    pthread_setaffinity_np(h, sizeof(cpu_set_t), &mask);
    ...
}

int main(int argc, char* argv[]){
    pthread_t h; int idH=1;

    pthread_create(&h, NULL, &funcionHilo, (void*)&idH);
    ...
}
```

¿Cómo sabemos cuál es la numeración de cada core?

- Intuitivamente, podemos suponer la siguiente identificación:



- Sin embargo, cada sistema operativo (y distribución o versión) identifica los cores de manera diferente.
- En Linux, esta identificación podemos obtenerla mediante el comando:

```
cat /proc/cpuinfo
```

- El recuadro muestra una salida simplificada de `cpuinfo` en una máquina con dos procesadores quad-core.
- Se pueden observar tres identificadores:
 - **Processor (ID):** identifica la unidad de procesamiento. Numeradas de 0 a 7
 - **Physical id:** identifica el socket del Processor. Numerados 0 y 1
 - **Core id:** identifica el core dentro de ese socket. Numerados 0 a 3 por socket
- La salida nos dice que la topología es la siguiente:



```
processor      : 0
physical id    : 0
core id       : 0
processor      : 1
physical id    : 1
core id       : 0
processor      : 2
physical id    : 0
core id       : 2
processor      : 3
physical id    : 1
core id       : 2
processor      : 4
physical id    : 0
core id       : 1
processor      : 5
physical id    : 1
core id       : 1
processor      : 6
physical id    : 0
core id       : 3
processor      : 7
physical id    : 1
core id       : 3
```


Posix Threads

Estructura de programa Pthreads

Definición de hilos

Creación de hilos

Función de comportamiento de hilos

Espera por finalización de hilos

Sincronización:

Mutexes

Variables condición

Barreras

Afinidad

Comentarios finales sobre Pthreads

- **Pthreads:**
 - Amplio control sobre la creación y el manejo de los hilos.
 - Tanto para paralelismo funcional como para paralelismo de datos.
 - Mas adecuada para el paralelismo funcional por su versatilidad.
 - Mayor esfuerzo de programación.

OpenMP

Modelo Fork-Join

Core elements:

Control de flujo paralelo

Entorno de datos y manejo de variables

Trabajo compartido y distribución de la carga

Sincronización

Funciones de biblioteca y variables de entorno

Afinidad

Comentarios finales sobre OpenMP

- **OpenMP¹** : API para programación paralela sobre memoria compartida multiplataforma
- Nace en 1997 para facilitar la **programación paralela** ante la disponibilidad de sistemas multiprocesador con memoria compartida (**SMP**).
- Consorcio (HP, IBM, AMD, Intel, SGI, ARM etc...)
- Se basa en directivas aplicadas a los lenguajes:
 - C/C++
 - Fortran
- Incluida en el compilador GCC:
 - Cabecera de programa: `#include<omp.h>`
 - Compilación: `gcc -fopenmp fuente.c -o ejecutable`
- **Runtime system** que gestiona el entorno de ejecución.

¹www.openmp.org

OpenMP

Modelo Fork-Join

Core elements:

Control de flujo paralelo

Entorno de datos y manejo de variables

Trabajo compartido y distribución de la carga

Sincronización

Funciones de biblioteca y variables de entorno

Afinidad

Comentarios finales sobre OpenMP

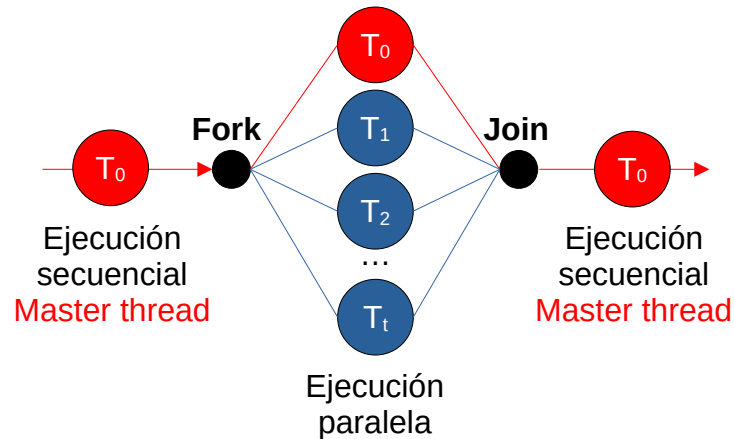
OpenMP

Modelo Fork-Join

63

- **Modelo Fork-Join de OpenMP:**

- Hilo principal (**master thread**) ejecuta código secuencial una región paralela.
- En ese punto, se crean varios hilos (**fork**), que ejecutan trabajo en paralelo.
- Al finalizar, los hilos se sincronizan (**join**) y la ejecución continúa secuencial.



- Cada hilo tiene un **identificador único** (thread id) numerados desde 0.
 - **Master thread** siempre es 0

OpenMP

Modelo Fork-Join

Core elements:

Control de flujo paralelo

Entorno de datos y manejo de variables

Trabajo compartido y distribución de la carga

Sincronización

Funciones de biblioteca y variables de entorno

Afinidad

Comentarios finales sobre OpenMP

OpenMP

Core Elements

65

- Funcionalidad organizada en **Core Elements**:
 - **Control de flujo paralelo**: Control de regiones paralelas y flujo de ejecución.
 - **Entorno de datos y manejo de variables**: Alcance de las variables.
 - **Trabajo compartido y distribución de la carga**: Distribución de iteraciones o secciones entre hilos
 - **Sincronización**: Coordinación entre hilos.
 - **Funciones de biblioteca y variables de entorno**: Control de ejecución externa al código.

Core elements

Control de flujo paralelo

parallel

Entorno de datos Manejo de variables

shared
private
firstprivate
lastprivate
default

Trabajo compartido Distribución de la carga

for
reduction
section
single
master
schedule

Sincronización

critical
atomic
ordered
barrier
nowait

Funciones de usuario Variables de entorno

Funciones:
omp_get_thread_num()
omp_set_num_threads()
omp_get_num_threads()

Variables:
OMP_NUM_THREADS
OMP_SCHEDULE

OpenMP

Modelo Fork-Join

Core elements:

Control de flujo paralelo

Entorno de datos y manejo de variables

Trabajo compartido y distribución de la carga

Sincronización

Funciones de biblioteca y variables de entorno

Afinidad

Comentarios finales sobre OpenMP

- En el caso específico del **lenguaje C**, se utiliza la directiva **#pragma**:

```
#pragma instrucción
```

- Da información adicional al compilador
 - Suprimir un mensaje de error específico
 - Chequear dependencias de archivos
 - OpenMP: interacción con el Runtime System

```
# pragma omp <directiva> [cláusula [ , ...] ...]
```

- No retorna errores ante directivas **#pragma** desconocidas.
 - El compilador la ignora.



- Estructura típica que crea una región paralela:

```
#include<omp.h>

int main(int argc,char* argv[]){

    #pragma omp parallel
    {
        "Región paralela"
    }

    return 0;
}
```

#pragma omp parallel crea varios hilos.
Cada hilo ejecuta el código dentro de las llaves.

```
#include<omp.h>

int main(int argc,char* argv[]){
    "Región secuencial S1"
    #pragma omp parallel
    {
        "Región paralela P1"
    }
    "Región secuencial S2"
    #pragma omp parallel
    {
        "Región paralela P2"
    }
    "Región secuencial S3"

    return 0;
}
```

Master thread (Id=0) ejecuta en forma secuencial.

Primera directiva **#pragma omp parallel**
El **Runtime System** crea T hilos (Id = 0 .. T-1) (**FORK**)
Cada hilo ejecuta en paralelo el código entre llaves.c

Hilos se esperan (**barrera implícita**) (**JOIN**)

Runtime System duerme los hilos
Master thread continúa de forma secuencial.

Siguiente directiva **#pragma omp parallel**
Runtime System despierta los T hilos (**FORK**)



Para el sistema operativo, el costo de crear hilos una y otra vez es mayor que dormirlos y despertarlos.

- Por defecto, OpenMP determina el número de cores y crea un hilo por cada core.
- Tres formas de hacerlo explícitamente:
 - Cláusula **num_threads**:

```
#include<omp.h>

int main(int argc,char* argv[]){
    int T=...;
    #pragma omp parallel num_threads(T)
    {
        "Región paralela"
    }
    return 0;
}
```

- Función **omp_set_num_threads**.
- Variable de entorno **OMP_NUM_THREADS**.

OpenMP

Modelo Fork-Join

Core elements:

Control de flujo paralelo

Entorno de datos y manejo de variables

Trabajo compartido y distribución de la carga

Sincronización

Funciones de biblioteca y variables de entorno

Afinidad

Comentarios finales sobre OpenMP

- Dentro de las regiones paralelas podemos tener variables compartidas entre los hilos y variables privadas de cada uno de ellos.
- Para gestionar el entorno de los datos, se agregan clausulas a las directivas:
 - shared
 - private
 - firstprivate
 - lastprivate
 - default

- Por defecto, las variables definidas fuera de la región paralela son compartidas por todos los hilos.
- Podemos indicarlo explícitamente con la cláusula **shared**.

```
int main(int argc, char* argv[]) {  
    int x = 0; // Variable compartida  
  
    #pragma omp parallel  
    {  
        printf("x = %d\n", x);  
    }  
}
```



```
int main(int argc, char* argv[]) {  
    int x = 0; // Variable compartida  
  
    #pragma omp parallel shared(x)  
    {  
        printf("x = %d\n", x);  
    }  
}
```

- Por defecto, variables definidas en una región paralela son privadas para cada hilo.
 - Cada hilo tiene su propia copia.
- Variables fuera de la región paralela pueden tratarse como privadas con la cláusula **private**. **No se inicializan y su valor no se mantiene fuera de la región paralela.**

```
int main(int argc, char* argv[]) {  
  
    #pragma omp parallel  
    {  
        int x; // Variable privada  
        ...  
        printf("x = %d\n", x);  
    }  
}
```

```
int main(int argc, char* argv[]) {  
    int x = 0; // Variable privada  
  
    #pragma omp parallel private(x)  
    {  
        printf("x = %d\n", x);  
    }  
}
```

- Ejemplo de salida utilizando la clausula **private**:

```
int j = 4;  
#pragma omp parallel for private(j)  
    for(a=0; a<3; a++){  
        j+=a;  
        printf("%d\n", j);  
    }  
  
printf("Final %d\n", j);
```

Se crea una copia de **j** para cada hilo.
No se inicializa.
El **valor inicial** es “**basura**”.
El valor final es el que tenía previo a la región paralela.



Posible salida de programa

```
4321  
0  
543243  
1234  
Final 4
```



- **firstprivate**: idem **private** pero el valor se inicializa con el valor original de la variable. El valor final es el que tenía previo a la región paralela.

```
int j = 4;
#pragma omp parallel for firstprivate(j)
    for(a=0; a<3; a++){
        j+=a;
        printf("%d\n", j);
    }

printf("Final %d\n", j);
```

Posible salida de programa

```
4
6
7
5
Final 4
```



- **lastprivate**: idem **private** el valor finaliza con el último valor calculado.

```
int j;  
#pragma omp parallel for lastprivate(j)  
    for(a=0; a<3; a++){  
        j=a;  
        printf("%d\n",j);  
    }  
  
printf("Final %d\n",j);
```

Posible salida de programa

```
3  
2  
0  
1  
Final 1
```



- **lastprivate**: no inicializa. Para inicializar lo usamos en conjunto con **firstprivate**.

```
int j = 0;
#pragma omp parallel for lastprivate(j) firstprivate(j)
    for(a=0; a<3; a++){
        j+=a;
        printf("%d\n", j);
    }

printf("Final %d\n", j);
```

Posible salida de programa

```
3
2
0
1
Final 1
```



- La cláusula **default(none)** permite eliminar todo comportamiento por defecto y obligar al programador a especificar explícitamente la visibilidad de cada variable.
 - El compilador genera un **error** si no se especifica shared o private.
 - Previene errores por variables compartidas accidentalmente.

```
int main(int argc, char* argv[]) {  
    int x = 0;  
  
    #pragma omp parallel default(none) shared(x)  
    {  
        "Región paralela"  
    }  
}
```

OpenMP

Modelo Fork-Join

Core elements:

Control de flujo paralelo

Entorno de datos y manejo de variables

Trabajo compartido y distribución de la carga

Sincronización

Funciones de biblioteca y variables de entorno

Afinidad

Comentarios finales sobre OpenMP

- **Cláusulas para distribuir la carga de trabajo** de las regiones paralelas entre hilos:
 - **for**: distribuye las iteraciones de una sentencia **for** entre hilos.
 - **schedule**: para elegir el tipo de asignación:
 - Estática (por defecto)
 - Dinámica
 - Guiada
 - **reduction**: reducción a un valor global a partir de un operador asociativo.
 - **sections**: asigna a hilos bloques de código disjunto e independiente.
 - **single**: bloque de código ejecutado por un único hilo. Barrera implícita al final del bloque.
 - **master**: bloque de código ejecutado por el **Master thread**. Sin barrera al final del bloque.

- **Cláusula for:** distribuye las iteraciones de la sentencia **for** proporcionalmente entre los hilos.

```
int main(int argc, char* argv[]){
    int a[N];

    #pragma omp parallel
    {
        #pragma omp for
        for (int i = 0; i < N; i++)
            a[i] = ...;
    }
    return 0;
}
```

Simplificado

```
int main(int argc, char* argv[]){
    int a[N];

    #pragma omp parallel for
        for (int i = 0; i < N; i++)
            a[i] = ...;

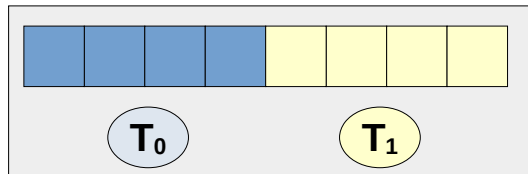
    return 0;
}
```



Por defecto, el **índice del for es privado** para cada hilo.
Barrera implícita al final de la sentencia for que involucra `#pragma omp for`

- Tres formas de distribuir iteraciones entre los hilos:
 - **Estática (por defecto):** proporcionalmente.
 - **Dinámica:** por demanda y de a bloques (**chunk**).
 - **Guiada:** dinámica con bloques variables por iteración.
- Agregamos la cláusula **`schedule`** a la directiva.

- **Distribución Estática (por defecto):** proporcionalmente.

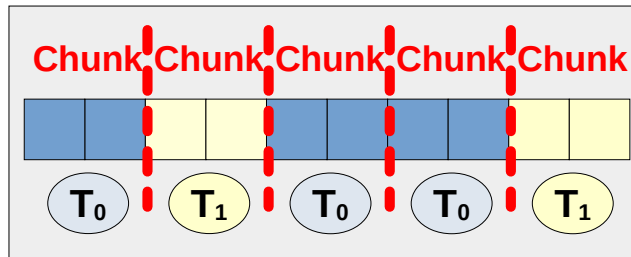


```
int main(int argc, char* argv[]) {  
    int a[N];  
  
    #pragma omp parallel for  
    for (int i = 0; i < N; i++) {  
        a[i] = ... ;  
    }  
}
```



```
int main(int argc, char* argv[]) {  
    int a[N];  
  
    #pragma omp parallel for schedule(static)  
    for (int i = 0; i < N; i++) {  
        a[i] = ... ;  
    }  
}
```

- **Distribución dinámica:** por demanda y de a bloques (**chunk**).

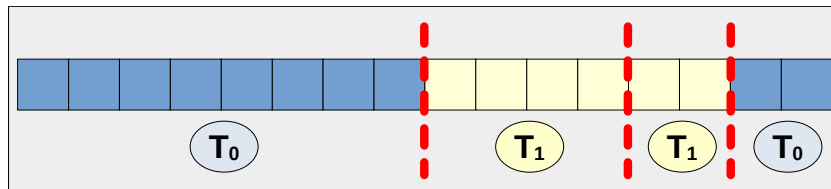


```
int main(int argc, char* argv[]) {  
    int a[N];  
  
    #pragma omp parallel for schedule(dynamic, 2)  
    for (int i = 0; i < N; i++) {  
        a[i] = ... ;  
    }  
}
```



Ideal cuando existe desbalance de carga.
Puede generar *overhead* de gestión.

- **Distribución guiada:** dinámica con bloques variables por iteración.



```
int main(int argc, char* argv[]) {  
    int a[N];  
  
    #pragma omp parallel for schedule(guided, 2)  
    for (int i = 0; i < N; i++) {  
        a[i] = ... ;  
    }  
}
```



- Un hilo ocioso solicita iteraciones.
- *Runtime system*: asigna un bloque igual al número de iteraciones restantes dividido el número de hilos.
- Nunca menor que chunk (en el ejemplo, 2).

- **Cláusula reduction:** reducción a un valor global a partir de un operador asociativo.

```
int main(int argc, char* argv[]) {  
    int suma = 0;  
    int a[N];  
  
    #pragma omp parallel for reduction(+:suma)  
    for (int i = 0; i < N; i++) {  
        suma += a[i];  
    }  
    printf("suma = %d\n", suma)  
}
```

Posibles operandos

+

-

*

&

|

^

&&

||

Max

Min



Se tiene una variable compartida (**suma**).

La cláusula **reduction** crea una copia privada (con firstprivate).

Cada hilo trabaja sobre su copia privada.

Al finalizar, se reducen los resultados en la variable compartida a partir del operando pasado a **reduction**.

- Es posible realizar más de una reducción en una misma iteración:
 - Mismo operador permite varias variables en la misma cláusula.

```
int sum1 = 0;
int sum2 = 0;
int arreglo[count];
...
int valor_maximo=arreglo[0];
...
#pragma omp parallel for reduction(+:sum1,sum2) reduction(max:valor_maximo)
for (int i = 0; i < count; ++i){
    sum1 += funcionA(arreglo[i]);
    sum2 += funcionB(arreglo[i]);
    if( arreglo[i] > valor_máximo){
        valor_maximo=arreglo[i];
    }
}
printf("%d %d %d\n",sum1,sum2,valor_maximo);
```



- **Cláusula sections:** asigna a hilos bloques de código disjunto e independiente.
 - Útil para Paralelismo funcional.

```
#pragma omp parallel
{
    #pragma omp sections {
        { //Código Sección 1 }
        #pragma omp section
        { //Código Sección 2 }
        #pragma omp section
        { //Código Sección 3 }
    }
}
```

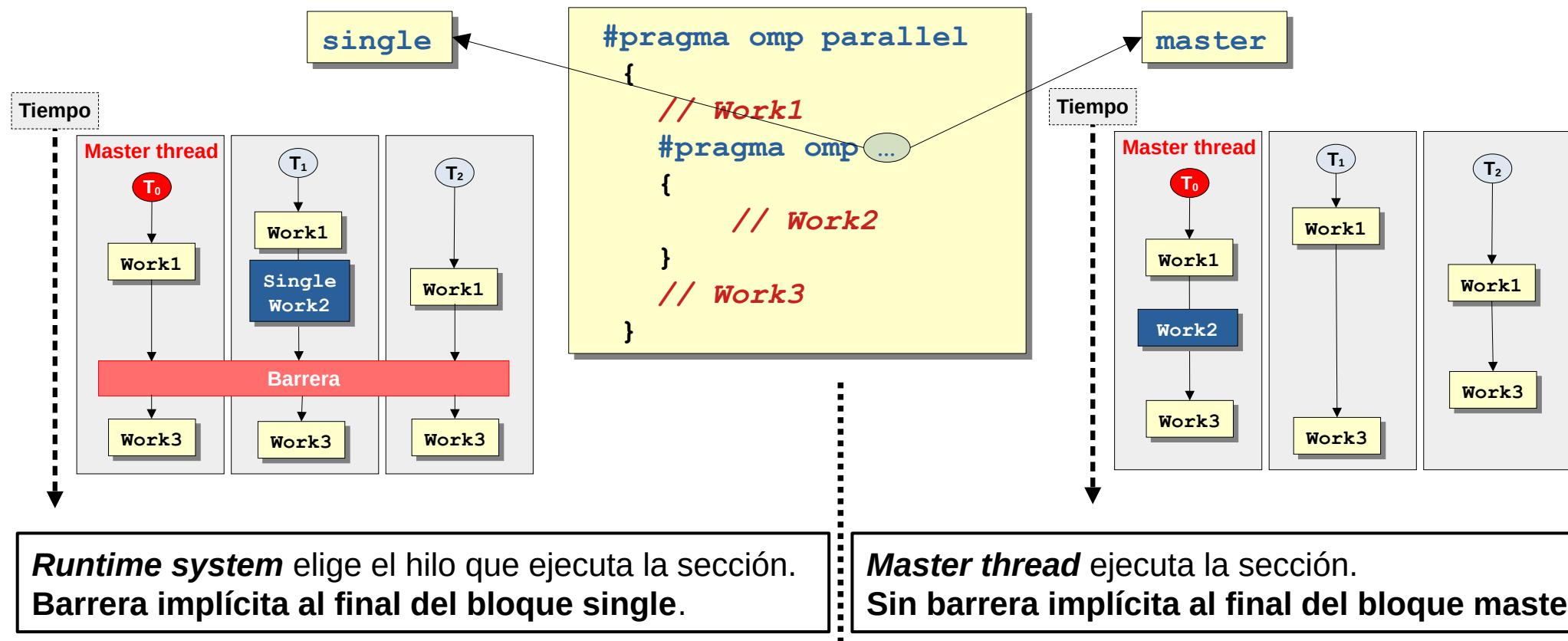
Simplificado

```
#pragma omp parallel sections
{
    { //Código Sección 1 }
    #pragma omp section
    { //Código Sección 2 }
    #pragma omp section
    { //Código Sección 3 }
}
```



Cada sección se ejecuta una sola vez por un único hilo.
Si hay más hilos que secciones, los hilos restantes esperan hasta que finalicen todas las secciones.
Barrera implícita.

- **Cláusula single y master:** bloque de código que será ejecutado por un único hilo.



OpenMP

Modelo Fork-Join

Core elements:

Control de flujo paralelo

Entorno de datos y manejo de variables

Trabajo compartido y distribución de la carga

Sincronización

Funciones de biblioteca y variables de entorno

Afinidad

Comentarios finales sobre OpenMP

- Cláusulas de mecanismos de sincronización:
 - **critical**: para secciones críticas en regiones paralelas.
 - **atomic**: ídem **critical**, solo para una operación sobre una variable compartida.
 - **ordered**: instrucciones en un **for** deben ejecutarse en orden de iteraciones.
 - **barrier**: barrera explícita.
 - **nowait**: modifica el comportamiento de otras directivas.

- **Cláusula critical:** encierra un bloque de código (**sección crítica**) que debe ser ejecutado por un hilo a la vez.
- Utilizado cuando no podemos beneficiarnos de la cláusula reduction:
 - Dado un arreglo de enteros, obtener el mínimo y la posición en que se encuentra.



```
int a[N];  
...  
int min_val;  
int min_pos = -1;  
  
#pragma omp parallel  
{  
    int local_min;  
    int local_pos = -1;  
    #pragma omp for  
    for (int i = 0; i < N; i++) {  
        if (local_pos == -1 || a[i] < local_min){  
            local_min = a[i];  
            local_pos = i;  
        }  
    }  
    #pragma omp critical  
    {  
        if (min_pos == -1 || local_min < min_val){  
            min_val = local_min;  
            min_pos = local_pos;  
        }  
    }  
}
```

OpenMP

Sincronización – atomic

94

- **Cláusula `atomic`:** ídem `critical`, solo para una operación sobre una variable compartida.
- Utilizado cuando no podemos beneficiarnos de la cláusula `reduction`:
 - Dado un arreglo de enteros, contar la cantidad de números negativos.



```
int a[N];
...
int negativos = 0;

#pragma omp parallel
{
    int local_negativos = 0;
    #pragma omp for
    for (int i = 0; i < N; i++) {
        if (a[i] < 0) {
            local_negativos++;
        }
    }
    #pragma omp atomic
    negativos += local_negativos;
}
```

OpenMP

Sincronización – ordered

95

- **Cláusula ordered:** instrucciones en un for deben ejecutarse en orden de iteraciones.

```
#pragma omp parallel for
  for (int i = 0; i < 3; i++)
    printf("%d\n",i);
```

Posible salida de programa

0
3
2
1

```
#pragma omp parallel for ordered
  for (int i = 0; i < 3; i++)
    printf("%d\n",i);
```

Posible salida de programa (ordered)

0
1
2
3



OpenMP

Sincronización – ordered

96

- **Ejemplo útil de ordered:** parte del cuerpo del for puede ejecutarse en paralelo y deben almacenarse los resultados de manera ordenada, según el orden de las iteraciones, hacia un dispositivo de almacenamiento.

```
int a_in[N];
int a_out[N];

#pragma omp parallel
{
    #pragma omp for schedule(static,1) ordered
    for (int i = 0; i < N; i++){

        a_out[i] = Procesar(a_in[i]);

        #pragma omp ordered
        FileWrite(FILE, v_out[i]);
    }
}
```

Cada hilo accede a posiciones consecutivas para maximizar el paralelismo.
Variante de schedule(static)

Región paralela



Salida secuencial ordenada

- **Cláusula barrier:** barrera explícita.

```
#pragma omp parallel {  
  
    //Ejecución paralela antes  
  
    #pragma omp barrier // Todos los hilos se esperan en este punto  
  
    //Ejecución paralela después  
}
```

- **Cláusula nowait:** modifica el comportamiento de otras directivas. En particular, **evita barreras implícitas.**

```
#pragma omp parallel{  
    #pragma omp for nowait  
    for(int i=0; i < N; i++){  
        // Procesamiento del for  
    }  
    // Esta línea puede ser alcanzada mientras otros hilos aún están ejecutando la iteración.  
    // Procesamiento fuera del for  
}
```

OpenMP

Modelo Fork-Join

Core elements:

Control de flujo paralelo

Entorno de datos y manejo de variables

Trabajo compartido y distribución de la carga

Sincronización

Funciones de biblioteca y variables de entorno

Afinidad

Comentarios finales sobre OpenMP

- **Funciones de biblioteca:** funciones de la API para control en el código.
 - `omp_get_num_threads()`: Retorna cuantos hilos que se crearon.
 - `omp_get_thread_num()`: Retorna el identificador del hilo.
 - `omp_get_num_procs()`: Retorna el número de unidades de procesamiento.
 - `omp_set_num_threads(int num_threads)`: Alternativa a la cláusula `num_threads`. Determina el número de hilos que se crearán.
 - `omp_set_max_active_levels(int nestedLevel)` y `omp_get_level()`: Paralelismo anidado.
- **Variables de entorno:** para control de ejecución fuera del código.
 - `OMP_NUM_THREADS`: ídem cláusula `num_threads` y función `omp_set_num_threads`
 - `OMP_SCHEDULE = [static, dynamic, guided]`
 - `OMP_MAX_ACTIVE_LEVELS`: ídem función `omp_set_max_active_levels(int nestedLevel)`

Funciones de biblioteca y variables de entorno – Anidamiento

- **Anidamiento:** cuando los hilos crean nuevos hilos.
- Ejemplo: se crean 2 hilos. Luego, cada hilo crea 4 hilos (8 hilos en total)

```
omp_set_max_active_levels(2);  
  
#pragma omp parallel num_threads(2)  
{  
    printf("Nivel %d\n", omp_get_level());  
    #pragma omp parallel num_threads(4)  
    {  
        printf("Nivel %d\n", omp_get_level());  
    }  
}
```



Possible salida:

ID hilo externo:0
ID hilo externo:1
ID hilo interno:0
ID hilo interno:3
ID hilo interno:1
ID hilo interno:2
ID hilo interno:0
ID hilo interno:1
ID hilo interno:2
ID hilo interno:3

Si omitimos `omp_set_max_active_levels` o el nivel de anidamiento no soporta lo solicitado, la creación interna se ignora!!!

Funciona si omitimos la función pero exportamos la variable de entorno `OMP_MAX_ACTIVE_LEVELS`

- Distribución runtime:

```
#pragma omp parallel for schedule(runtime)
for(int i=0; i<N; i++)
    "Código"
```

Cambiar la política de planificación sin necesidad de compilar el código.
Desde el Sistema Operativo modificamos la variable de entorno `OMP_SCHEDULE`.
En Linux:

Estática: `export OMP_SCHEDULE="static"`

Dinámica: `export OMP_SCHEDULE="dynamic,4"`

Guiada: `export OMP_SCHEDULE="guided,2"`

OpenMP

Modelo Fork-Join

Core elements:

Control de flujo paralelo

Entorno de datos y manejo de variables

Trabajo compartido y distribución de la carga

Sincronización

Funciones de biblioteca y variables de entorno

Afinidad

Comentarios finales sobre OpenMP

- Control de afinidad mediante cláusulas y/o variables de entorno:
 - **Cláusula `proc_bind` (*Binding*)** o **variable de entorno `OMP_PROC_BIND`**: distribución bajo determinadas políticas.
 - `close` | `spread` (por defecto) | `master` 
 - Variable de entorno `OMP_PLACES`: Asignación personalizada
 - Lista de processors Id
 - En conjunto con ***binding***: `cores` (por defecto)
 - Alternativas con Hyperthreading: `sockets` | `threads` 



OpenMP

Afinidad – Binding

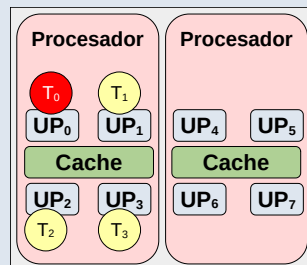
104

- Políticas de afinidad: cláusula **proc_bind** o variable de entorno **OMP_PROC_BIND**

```
#pragma omp parallel proc_bind(master) {  
    ...  
}
```

OMP_PROC_BIND=master

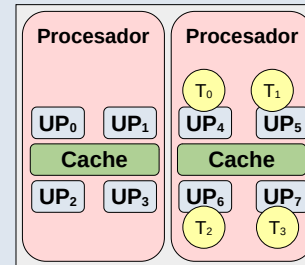
**Cercanos
Al master thread**



```
#pragma omp parallel proc_bind(close) {  
    ...  
}
```

OMP_PROC_BIND=close

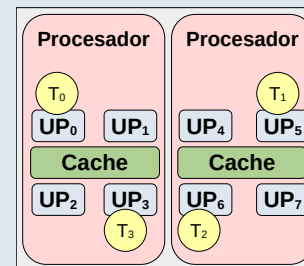
**Cercanos
Comparten cache**



```
#pragma omp parallel proc_bind(spread) {  
    ...  
}
```

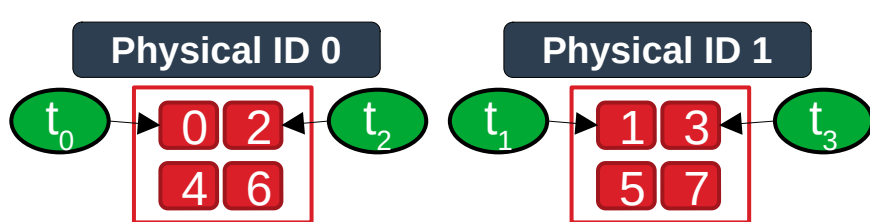
OMP_PROC_BIND=spread

**Dispersos
Reduce contención**



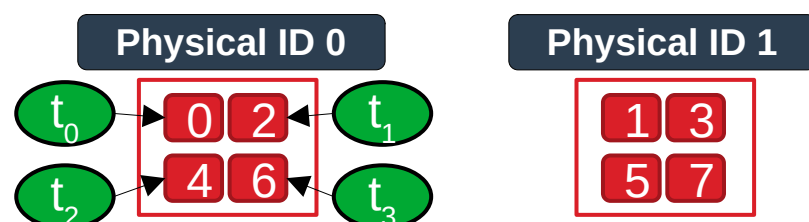
Afinidad – OMP_PLACES – Lista de Processors Id

- A partir de la variable de ambiente `OMP_PLACES`, podemos pasar una lista de processors Id donde ubicar los hilos.
- El thread Id coincide con el índice de la lista y el contenido con los Processor Id.
- Suponiendo 4 hilos, dos formas de ubicar los hilos sobre la siguiente arquitectura:



`OMP_PLACES='{0},{1},{2},{3}'`

Equivalente: `OMP_PLACES='{0}:4'`



`OMP_PLACES='{0},{2},{4},{6}'`

Equivalente: `OMP_PLACES='{0}:4:2'`



OpenMP

Modelo Fork-Join

Core elements:

Control de flujo paralelo

Entorno de datos y manejo de variables

Trabajo compartido y distribución de la carga

Sincronización

Funciones de biblioteca y variables de entorno

Afinidad

Comentarios finales sobre OpenMP

- **OpenMP:**
 - Control en la creación y el manejo de los hilos transparente al usuario (Runtime System).
 - Pensado para paralelismo de datos
 - Menor esfuerzo de programación.

Próxima clase

Capítulo 7

Modelo de programación de memoria distribuida.

Capítulo 8

Modelo de programación híbrido.

